

Week 02 Lab – Image Processing

Name: Zeeshan Ahmed Roll Number: 50

Week 02 Lab: Image Processing Through a Four-Step Pipeline

Course: Perceptual Computing / Computer Graphics and Human-Computer Interaction

Coverage: Week 1-3 lecture material

Work mode: Individual submission

Student Information

- **Name:** Zeeshan Ahmed

- **Roll number:** 50

- **GitHub username:** must be `2K24CSE[roll\_number]` (example: `2K24CSE105`).

- **GitHub repository:** create one **public** repository named exactly `CGHCI` on that account. Use this same repository for all labs this semester.

This lab is designed to make you practice turning image-processing rules into reliable Python programs. Complete **all five problems** and preserve your formulas, manual traces, implementation, tests, and short explanations.

Four Simple Steps

Use these same four steps for each problem:

1. **Inputs and output:** What information goes in? What should come out?
2. **Rule:** Write the formula or condition.
3. **Small example:** Solve one example by hand.
4. **Python:** Complete the code and test it.

Write your answers under the headings `Step 1`, `Step 2`, `Step 3`, and `Step 4`. The two samples above show exactly what to do.

How to Complete This Lab

- Use the examples as a guide.
- Complete all five problems.
- Do not worry about advanced Python. Use the starter function.
- Each problem has only 2 TODOs. Fill them in, then run the cell.
- Write a short 1-2 line answer for Steps 1-3 before Step 4 code.

Important: Keep image values between `0` and `255`.

Worked Sample 1: RGB to Grayscale

A color pixel has red, green, and blue components. Compute one grayscale luminosity value.

Step 1: Inputs and outputs

- Inputs: integers `r`, `g`, `b`, each in `[0, 255]`.
- Output: one integer in `[0, 255]`.

Step 2: Mathematical rule

$$Y = 0.299R + 0.587G + 0.114B$$

Step 3: Manual trace

For `(R, G, B) = (180, 120, 60)`:

$$Y = 0.299(180) + 0.587(120) + 0.114(60) = 131.1$$

Rounding gives `131`. The code cell shows how to turn this trace into a tested function.

Python Code

```
def rgb_to_grayscale(r: int, g: int, b: int) -> int:
    """Convert one RGB pixel to a rounded grayscale luminosity value."""
    luminosity = 0.299 * r + 0.587 * g + 0.114 * b
    return int(round(luminosity))

sample_result = rgb_to_grayscale(180, 120, 60)
print("Sample result:", sample_result)
assert sample_result == 131

# Extra test: equal channels should remain unchanged.
assert rgb_to_grayscale(80, 80, 80) == 80
print("Sample 1 passed.")
```

Output / Test Result

```
Sample result: 131
Sample 1 passed.
```

Worked Sample 2: Brightness and Contrast Adjustment

Given a grayscale image `I`, adjust contrast by `alpha` and brightness by `beta`.

Step 1: Inputs and outputs

- Input: a 2D grayscale NumPy array with values in `[0, 255]`.
- Parameters: `alpha > 0` and numeric `beta`.
- Output: same shape, `uint8`, with values in `[0, 255]`.

Step 2: Mathematical rule

$$O(x,y) = \alpha I(x,y) + \beta$$

Then clip each value to the valid image range:

$$O_{\text{final}}(x,y) = \min(\max(O(x,y), 0), 255)$$

Step 3: Manual trace

For `I = [[100, 150], [200, 50]]`, `alpha = 1.5`, and `beta = -20`:

- `100` -> `130`
- `150` -> `205`
- `200` -> `280` -> `255` after clipping

- `50 -> 55`

Expected output: `[[130, 205], [255, 55]]`.

Python Code

```
import numpy as np

def adjust_brightness_contrast(image: np.ndarray, alpha: float, beta: float) -> np.ndarray:
    """Apply  $O = \alpha * I + \beta$  and clip the result to  $[0, 255]$ ."""
    image_float = image.astype(np.float32)
    adjusted = alpha * image_float + beta
    return np.clip(adjusted, 0, 255).astype(np.uint8)

sample_image = np.array([[100, 150], [200, 50]], dtype=np.uint8)
sample_output = adjust_brightness_contrast(sample_image, alpha=1.5, beta=-20)
expected_output = np.array([[130, 205], [255, 55]], dtype=np.uint8)
np.testing.assert_array_equal(sample_output, expected_output)
print(sample_output)

# Extra test: a large positive beta must clip at 255.
extra = adjust_brightness_contrast(np.array([[240]], dtype=np.uint8), alpha=1.0, beta=30)
assert extra[0, 0] == 255
print("Sample 2 passed.")
```

Output / Test Result

```
[[130 205]
 [255  55]]
Sample 2 passed.
```

Five Problems - 1 Mark Each

The first two sections are solved examples. Complete these five short problems:

1. **Thresholding:** change a grayscale image into black and white.
2. **Memory:** calculate image memory in bytes.
3. **Mean filter:** find the average of 9 pixels.
4. **Contrast:** change values from one range to another.
5. **Sobel:** find the strength of an edge.

For each problem, write Steps 1-3 and complete the `TODO` items in the code cell.

Python Code

```
import numpy as np

# TODO 1: Return 255 when a pixel is >= threshold; otherwise return 0.
def threshold_image(image: np.ndarray, threshold: int) -> np.ndarray:
    """Convert a grayscale image to black and white."""
    return np.where(image >= threshold, 255, 0).astype(np.uint8)

problem_1_input = np.array([[20, 128, 200], [100, 150, 250]], dtype=np.uint8)
problem_1_expected = np.array([[0, 255, 255], [0, 255, 255]], dtype=np.uint8)

# TODO 2: Remove the # symbols and run the test.
np.testing.assert_array_equal(threshold_image(problem_1_input, 128), problem_1_expected)
print("Problem 1 passed")
```

Output / Test Result

```
Problem 1 passed
```

Problem 1: Thresholding (1 mark)

Turn each grayscale value into black (`0`) or white (`255`).

Step 1: Inputs and output

Input: an image and a threshold number. Output: an image of only `0` and `255`.

Step 2: Rule

If `pixel >= threshold`, output `255`. Otherwise output `0`.

Step 3: Small example

For threshold `128`:

```
```text
20 128 200
100 150 250
```
```

Answer: `0 255 255` / `0 255 255`.

Step 4: Python

Fill in the 2 `TODO` items in the next cell. Note: `128` counts as white.

Problem 2: Image Memory (1 mark)

Calculate how many bytes an uncompressed image uses.

Step 1: Inputs and output

Inputs: width, height, and bits per pixel (`bpp`). Output: memory in bytes.

Step 2: Rule

`bytes = width \* height \* bpp / 8`

Step 3: Small example

For `1920 x 1080` and `24 bpp`: `1920 \* 1080 \* 24 / 8 = 6,220,800` bytes.

Step 4: Python

Fill in the 2 `TODO` items.

Python Code

```
# TODO 1: Multiply width, height, and bpp, then divide by 8.
def display_memory_bytes(width: int, height: int, bpp: int) -> int:
    """Return image memory in bytes."""
    return width * height * bpp // 8

# TODO 2: Remove the # symbols and run the given test.
expected_bytes = 1920 * 1080 * 24 // 8
assert display_memory_bytes(1920, 1080, 24) == expected_bytes
print("Problem 2 passed")
```

Output / Test Result

Problem 2 passed

Problem 3: Average 9 Pixels (1 mark)

Find the average of a small `3 x 3` image block.

Step 1: Inputs and output

Input: nine pixel values in a `3 x 3` array. Output: one rounded number.

Step 2: Rule

`average = sum of all 9 values / 9`

Step 3: Small example

```
```text
10 20 10
30 50 30
10 20 10
```
```

Sum = 190, so `190 / 9 = 21` (rounded).

Step 4: Python

Fill in the 2 `TODO` items.

Python Code

```
# TODO 1: Add all 9 values, divide by 9, and round the answer.
def mean_filter_3x3(neighborhood: np.ndarray) -> int:
    """Return the rounded average of 9 pixels."""
    return int(round(np.sum(neighborhood) / 9))

problem_3_input = np.array([[10, 20, 10], [30, 50, 30], [10, 20, 10]], dtype=np.uint8)

# TODO 2: Remove the # symbols and run the given test.
assert mean_filter_3x3(problem_3_input) == 21
print("Problem 3 passed")
```

Output / Test Result

Problem 3 passed

Problem 4: Change Image Contrast (1 mark)

Change numbers from one range to another range.

Step 1: Inputs and output

Input: an array and two ranges (old and new). Output: an array with the new values.

Step 2: Rule

`new = (value - old\_min) / (old\_max - old\_min) \* (new\_max - new\_min) + new\_min`

Step 3: Small example

Change `[50, 100, 150]` from range `[50, 150]` to `[0, 255]`:

`50 -> 0`, `100 -> 127.5`, `150 -> 255`.

Step 4: Python

Fill in the 2 `TODO` items.

Python Code

```
# TODO 1: Use the formula from the problem description, then np.clip to stay in [out_min, out_max].
def contrast_stretch(
    image: np.ndarray,
    in_min: float,
    in_max: float,
    out_min: float = 0.0,
    out_max: float = 255.0,
) -> np.ndarray:
    """Change image values from one range to another."""
    stretched = (image.astype(np.float32) - in_min) / (in_max - in_min)
    stretched = stretched * (out_max - out_min) + out_min
    return np.clip(stretched, out_min, out_max)

problem_4_input = np.array([50, 100, 150], dtype=np.float32)

# TODO 2: Remove the # symbols and run the given test.
expected = np.array([0.0, 127.5, 255.0], dtype=np.float32)
np.testing.assert_allclose(contrast_stretch(problem_4_input, 50, 150), expected)
print("Problem 4 passed")
```

Output / Test Result

Problem 4 passed

Problem 5: Find an Edge with Sobel (1 mark)

Use a Sobel filter to find how strong an edge is in the middle of a `3 x 3` block.

Step 1: Inputs and output

Input: a `3 x 3` grayscale block. Output: `gx`, `gy`, and edge strength.

Step 2: Rule

Multiply the block by each kernel below, add the 9 results to get `gx` and `gy`:

```
```text
Gx = Gy =
-1 0 1 -1 -2 -1
-2 0 2 0 0 0
-1 0 1 1 2 1
```
```

Then `strength = sqrt(gx \* gx + gy \* gy)`.

Step 3: Small example

```
```text
20 20 200
```

20 20 200

20 20 200

...

`gx = 720`, `gy = 0`, so `strength = 720`.

## Step 4: Python

Fill in the 2 `TODO` items.

## Python Code

```
TODO 1: Write the two kernels, multiply them with the block, and add the results to get gx and gy.
def sobel_response(block: np.ndarray) -> tuple[float, float, float]:
 """Return gx, gy, and edge strength."""
 gx_kernel = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=np.float32)
 gy_kernel = np.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]], dtype=np.float32)
 gx = float(np.sum(block * gx_kernel))
 gy = float(np.sum(block * gy_kernel))
 strength = float(np.sqrt(gx * gx + gy * gy))
 return gx, gy, strength

problem_5_input = np.array([[20, 20, 200], [20, 20, 200], [20, 20, 200]], dtype=np.float32)

TODO 2: Remove the # symbols and run the given test.
gx, gy, strength = sobel_response(problem_5_input)
assert gx == 720.0 and gy == 0.0
print("Problem 5 passed")
```

## Output / Test Result

Problem 5 passed

## Submission Checklist and Marks

### Submit

1. Your GitHub username must be `2K24CSE[roll\_number]` (example: `2K24CSE105`).
2. Create one **public** repository on that account named exactly `CGHCI` (reuse it for every lab this semester). Keep it public - do not make it private.
3. Put this week's work inside a folder named `week02`.
4. Submit this `.ipynb` file or one or more `.py` files.
5. Include your name, roll number, Steps 1-4, and working tests.

### Marks: 5 total

| Problem | Marks |

|---|---|

| Problem 1: Thresholding | 1 |

| Problem 2: Image memory | 1 |

| Problem 3: Average 9 pixels | 1 |

| Problem 4: Change contrast | 1 |

| Problem 5: Sobel edge | 1 |

**\*\*Do not submit screenshots only. Submit the code file or notebook.\*\***